

LearnSphere

A learning management system where students take courses module by module, hand in assignments and follow their progress, and instructors build the courses and mark the work.

PROJECT	Learning Management System (LMS): LearnSphere
DEVELOPER	Abhyuday Saxena
INSTITUTION	Thapar Institute of Engineering and Technology
DOMAIN	Full Stack Development · Individual Major Project
FRONTEND	React 18, Vite 5, React Router 6, custom CSS
BACKEND	Node.js, Express 4, Firebase Admin SDK, Helmet
DATABASE	Supabase (PostgreSQL) with Row Level Security
AUTHENTICATION	Firebase Authentication: email/password and Google sign-in
HOSTING	Vercel (frontend) · Render (backend) · Supabase (database)

LINKS

Live application: <https://quillancemaajorproject1-client.vercel.app>

Source code: https://github.com/abhyudaysaxena06/QuillanceMaajorProject_1

Problem statement and objectives

In most of my own courses, the reading material sat in a Drive folder, deadlines were announced in a WhatsApp group and marks came back in a spreadsheet. It was easy to miss something that was due, and the instructor had no simple way to see who had fallen behind.

LearnSphere puts all of this in one website. Students, instructors and admins each get their own pages, and everything (courses, submissions, marks and progress) is stored in one database behind one REST API.

Objectives

- Secure registration and login, with role-based access for students, instructors and admins.
- Instructors can create, edit and delete courses, organise them into ordered modules and publish assignments.
- Students can browse the catalog, enroll, study module material and submit assignments.
- Progress is calculated from completed modules and shown as percentages, progress bars and status.
- Instructors review submissions, award marks, write feedback or request a resubmission.
- Build and deploy every layer myself: frontend, API, login, database and hosting.

Scope against the minimum completion criteria

REQUIREMENT FROM THE BRIEF	STATUS	WHERE
Login and registration	Done	Email/password and Google via Firebase
Student / Admin roles	Done	student, instructor, admin
Course management	Done	Instructor course editor
Course enrollment	Done	Course page, unique per student
Course modules and learning materials	Done	Ordered modules with notes, video and links
Assignment management and submission	Done	Text, GitHub, Drive or project URL
Progress tracking	Done	Per module, per course and overall
Student and admin dashboards	Done	Role-specific dashboards
Database integration	Done	Supabase PostgreSQL, 7 tables
Optional features from the brief	Done	Quizzes, certificates, email notifications, search and filtering, discussion forum, profile photo upload, password reset, analytics charts

User roles and access control

Hiding a button in the interface is not enough, so every one of these rules is also checked by the server.

ROLE	CAPABILITIES	MAIN SCREENS
Student	Register and log in, browse and filter courses, enroll or leave, study modules, pass module quizzes, submit text, links or files until graded, view marks and feedback, ask questions in the course discussion, keep personal notes, track progress and earn certificates.	Dashboard, Catalog, My Courses, Module view, Assignments, Profile
Instructor	Create, edit, publish and delete their own courses; add, reorder and delete modules with learning materials; create assignments with deadlines and maximum marks; review submissions; give marks and feedback or request a resubmission; view the roster with each student's progress; build module quizzes; export the gradebook as CSV; send announcements; answer course discussions.	Dashboard, Manage Courses, Course Editor, Submissions
Admin	Everything an instructor can do, on every course. Also: view all users, change roles, activate or deactivate accounts, transfer course ownership, send platform announcements and see platform statistics.	Dashboard, Users, all instructor screens

How a request is authorised

1. The user signs in with Firebase (email/password or Google). Firebase stores the password as a salted hash and returns a short-lived signed ID token (a JWT).
2. The React client sends that token with every API call in the `Authorization: Bearer` header.
3. The `authenticate` middleware verifies the token with the Firebase Admin SDK, then loads the matching row from the `users` table, creating it on first login. Deactivated accounts are rejected with 403.
4. `requireRole()` checks the role for the route, and ownership helpers (`assertCanManage`, `isEnrolled`) check that the user owns the course or is enrolled in it.

The browser never talks to the database. Row Level Security is switched on for every table and no public policies exist, so the public Supabase key cannot read anything. Only the server, which holds the service role key, can query the data.

Onboarding

On first login a new user chooses *I want to learn* or *I want to teach*. Emails listed in the server's `ADMIN_EMAILS` setting are made admins automatically. Admins can change any user's role later.

Functional modules

Authentication

- Registration with name, email and password, with form validation
- Clear errors for a wrong password or an already-registered email
- Google sign-in and password reset by email
- Protected routes and logout

Student dashboard

- Enrolled and completed courses, modules done, pending work, average marks
- Per-course progress bars
- Upcoming deadlines
- Activity log of finished modules, submissions and grades

Course management

- Create, edit, delete; draft or published
- Title, description, category, difficulty, duration, image
- Catalog with search and category filter

Enrollment

- One-click enroll and leave
- Duplicate enrollment blocked by the API and a unique constraint
- Enrollment date, progress percentage and status

Modules and materials

- Ordered modules, reordered with up/down controls
- Notes, embedded YouTube/Vimeo video, duration
- Links to notes, PDFs, videos, source code, references and exercises

Assignments

- Instructions, deadline, maximum marks
- Submit text and/or a GitHub, Drive or project link
- Update until graded; late submissions flagged

Review and marks

- Central Submissions page: awaiting review, graded, resubmission requested
- Marks validated against the maximum, plus written feedback
- Status: submitted, graded or resubmit

Progress and admin

- Segmented progress bar, one segment per module
- Instructor roster with each student's progress
- Admin user list: role changes, activation, enrollment counts

Learning, community and analytics

Module quizzes

- Multiple-choice questions with explanations, built in the module editor
- Graded on the server; correct answers are never sent before submitting
- A module with a quiz is completed by reaching its pass mark
- Every attempt is stored with its score

Certificates

- Issued automatically when every module is complete
- Unique credential ID (LS-XXXXXXX)
- Public verification page at `/verify/:id`, no login needed
- Print-ready layout, save as PDF from the browser

File submissions

- PDF, ZIP, DOCX, PNG or JPG up to 10 MB
- Type and size checked on the server
- Stored in a private Supabase Storage bucket
- Downloaded through short-lived signed links

Course discussion

- Questions and replies per course
- Upvotes to surface common questions
- Instructor can mark the accepted answer
- Authors and instructors can delete threads

Notifications

- Bell with unread count in the navigation
- New assignment, marks, resubmission request, certificate, replies, announcements
- Optional email copies through any SMTP provider

Analytics and export

- 30-day enrollment trend, learners per course
- Submission breakdown and completion rate
- Gradebook CSV: progress, completion, certificate, marks per assignment

Personal tools

- Course notes with quick snippets, auto-saved, export to .txt
- Profile photo upload

Administration

- Announcements to a course or the whole platform
- Transfer course ownership between instructors
- API rate limiting and security headers

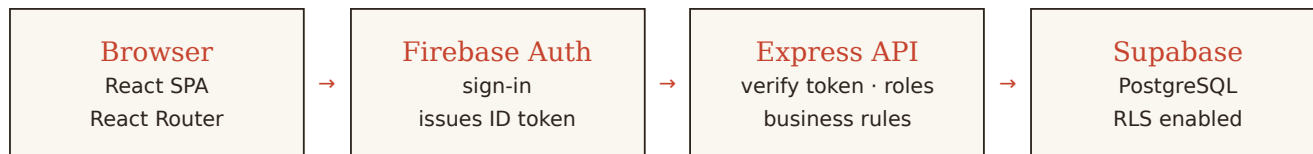
User workflows

Student: Register → Login → Browse courses → Enroll → Open course → Study modules → Mark complete → View assignments → Submit → Track progress → Course completed.

Instructor: Login → Dashboard → Create course → Add modules → Publish assignment → Monitor students → Review submissions → Give marks and feedback → Track progress.

System architecture

The React app only ever talks to the Express API, and the API is the only thing that talks to the database.



Project structure

```

client/          React frontend (Vite)
  src/api.js      fetch wrapper, attaches the Firebase ID token
  src/context/    AuthContext: session, profile, login/register/logout
  src/components/ Layout, CourseCard, ProgressBar, QuizPanel, Discussion,
                  TurnInForm, NotificationBell, Charts and others
  src/pages/      Home, Login, Onboarding, Dashboard, Courses, CourseDetail,
                  LessonView, MyLearning, Assignments, AssignmentDetail,
                  Teach, CourseEditor, Submissions, Admin, Profile, Certificate
server/          Express backend
  src/config/     Firebase Admin and Supabase clients
  src/middleware/ authenticate, requireRole
  src/routes/     auth, courses, lessons, assignments, discussions,
                  notifications, dashboard, admin
  src/lib/        notifications, file storage, progress and certificates
  scripts/seed-demo.js creates demo accounts and sample progress
database/        setup_all.sql (all tables and sample data), ER diagram
  
```

Design decisions

- I used Firebase for logins because it already handles password hashing, Google sign-in and reset emails, and Supabase for the data because the course data is relational and fits PostgreSQL well.
- Progress is not saved as a number. It is worked out from `lesson_progress` each time (finished modules divided by total modules), so it stays right when an instructor adds or removes a module.
- Each route copies only the fields it expects from the request body before writing anything, so extra fields sent by a client are ignored.
- Helmet adds the standard security headers, CORS only accepts requests from my own frontend, and there is a rate limit on the API.

Interface

I wanted the site to look like a printed course book rather than a typical admin dashboard, so it uses a cream background, dark brown text, a single terracotta accent and serif headings, with the word *विद्या* (knowledge) as a faint watermark. On phones the top menu turns into a full-screen menu, and the layout works down to about 360 px wide.

Database schema

There are eleven tables and two file storage buckets. Every table uses UUID keys, and deleting a course also deletes its modules, assignments and enrollments.

TABLE	KEY FIELDS	CONSTRAINTS
users	firebase_uid, email, name, role, onboarded, is_active, bio	firebase_uid unique; role in student / instructor / admin
courses	title, description, category, level, duration, thumbnail_url, published, instructor_id	instructor_id → users; level check
lessons (modules)	course_id, title, content, video_url, position, resources (JSON), quiz (JSON), pass_mark	course_id → courses; indexed by (course_id, position)
enrollments	user_id, course_id, enrolled_at, completed_at, certificate_id	unique (user_id, course_id); certificate_id unique
lesson_progress	user_id, lesson_id, completed_at	unique (user_id, lesson_id)
assignments	course_id, title, description, due_date, max_points	max_points > 0
submissions	assignment_id, student_id, content, link_url, file_path, status, grade, feedback	unique (assignment_id, student_id); status check
quiz_attempts	user_id, lesson_id, score, passed, answers	indexed by (user_id, lesson_id)
discussions / replies	course_id, user_id, title, body, upvotes · discussion_id, body, is_instructor_answer	cascade on course / thread delete
notifications	user_id, type, title, message, link, is_read	indexed by (user_id, is_read, created_at)

REST API

These are the main routes. Apart from the public ones, every request needs a valid Firebase token.

METHOD	ENDPOINT	PURPOSE	ACCESS
GET	/api/auth/me	Current profile (created on first login)	Signed in
GET	/api/courses	Catalog with search and category	Signed in
POST	/api/courses	Create course	Instructor
POST	/api/courses/:id/enroll	Enroll in a course	Student
POST	/api/lessons/:id/quiz	Submit quiz answers, graded on server	Enrolled
POST	/api/assignments/:id/submit	Submit or resubmit work	Enrolled
PATCH	/api/assignments/submissions/:id/grade	Marks, feedback, status	Owner
GET	/api/public/certificates/:id	Verify a certificate	Public
GET	/api/notifications	Notifications and unread count	Signed in
PATCH	/api/admin/users/:id	Change role, activate or deactivate	Admin

The full list of 45 endpoints is in docs/API.md. Errors are returned as { "error": "message" } with status 400, 401, 403, 404, 409 or 500.

Setup, deployment and demo accounts

Local setup

```
# 1. clone and install
git clone https://github.com/abhyudaysaksena06/QuillanceMaajorProject_1.git
cd QuillanceMaajorProject_1
npm run setup

# 2. database: run database/setup_all.sql in the Supabase SQL Editor
# 3. fill in server/.env and client/.env (see the .env.example files)

# 4. demo accounts and sample progress
npm run seed:demo

# 5. start API (port 5000) and web app (port 5173) together
npm run dev
```

Production deployment

- **Frontend** on Vercel (root client, Vite preset). vercel.json rewrites every path to index.html so client-side routes survive a refresh.
- **Backend** on Render (root server, npm start). Secrets are stored as Render environment variables.
- **Database** on Supabase; **authentication** on Firebase, with the Vercel domain added to Authorized domains.

Demo accounts

ROLE	EMAIL	PASSWORD	PREPARED DATA
Student	student@learnsphere.demo	Demo@1234	2 courses, passed quizzes, a certificate, a graded assignment
Student	student2@learnsphere.demo	Demo@1234	1 course, a submission awaiting review
Instructor	instructor@learnsphere.demo	Demo@1234	Owns the 3 sample courses
Admin	admin@learnsphere.demo	Demo@1234	User management and statistics

Testing

I tested everything by hand, following the checklist in the brief (the full list is in docs/TESTING.md). This covered correct and wrong logins, signing up twice with the same email, a student trying to open instructor and admin pages or call their APIs (they get a 403), creating, editing and deleting courses and assignments, enrolling twice (409), submissions being saved, progress updating after a module, form validation, the phone layout, and data still being there after restarting the server.

Outcome and future work

LearnSphere is live and covers everything the brief asked for, plus most of the optional features. Instructors and admins manage courses, modules, quizzes and assignments, and students enroll, finish modules, hand in work and get marks, feedback and certificates. Building it meant getting a React frontend, an Express API, Firebase logins, a PostgreSQL database and two hosting services to work together.

Challenges

- Login lives in Firebase but the data lives in Supabase. The server checks the Firebase token on every request and links each Firebase user to a row in my `users` table.
- A hosted database is reachable from the internet, so I turned on Row Level Security and kept the service key on the server only.
- A stored progress percentage went wrong whenever a module was added, so progress is now calculated from the finished modules instead.
- After deploying, the browser blocked calls from Vercel to Render. A CORS allow-list read from an environment variable fixed it.
- The quiz answers must not reach the browser, so the server removes them before sending a module to a student and marks the answers itself.

Future improvements

- Course ratings and reviews
- Live classes and attendance tracking
- Real-time notifications over WebSockets instead of polling
- Automated tests that run on every push

Submission contents

ITEM	LOCATION
Frontend and backend source	<code>client/</code> , <code>server/</code>
Database schema and sample data	<code>database/</code>
README and project documentation	<code>README.md</code> , <code>docs/</code> , this PDF
Output screenshots	<code>Screenshots/</code>
1-minute demonstration video	Video file in the submission folder
Deployment link	Cover page and README